YOUR NEXT MAP STARTS HERE.

TileForge Studio field guide

A complete reference for loading Emerald: Legends, editing maps and graphics, saving drafts, sharing the required files and getting a tested game build.

LOAD / STUDIO / EXPORT

For the quickest first session: load Legends, make one small map edit, check movement, save a draft, then export the edits ZIP and a portable project backup.

1. Choose the right starting point

2. Load, orient yourself and make a tiny first edit

3. Find the right tile and paint the map

4. View and edit collision and elevation

5. Change the map border or tileset pixels

6. Add or edit NPCs, signs, triggers and warps

7. Save drafts, undo mistakes and make backups

8. Export and send the required files back to chat

9. Integrate, build and check the result

10. Troubleshooting and next improvements

Online guide and downloads

Version 0.5.0 alpha · By 8BitTrade

1. Choose the right starting point

For Emerald: Legends maps, use **source mode**. TileForge edits the project's map, graphics and event files. You export those changes, then build the game project to produce a new ROM. Opening an existing `.gba` instead gives you manual graphics and hex editing; it does not unlock source maps or decompile the game.

What you have	What to open
The published Legends project, with no unpublished local edits	On the Load screen, choose **Emerald: Legends**. This reads the current public source and pins this session to that commit.
A Legends checkout with your own Porymap changes	Make a source pack from that checkout, then choose **Open your files**. This is the way to keep your local changes as the starting point.
Another compatible pokeemerald-style project	Open its source ZIP or a source pack. Standard Emerald and FRLG layout splits are supported.
A previously exported TileForge project ZIP	Open that ZIP to continue its files, original baselines, pins and custom groups.
A draft on this device	Tap its entry under saved drafts on the Load screen.
Only a `.gba` file	Use the ROM workspace for known-offset uncompressed 4bpp or hex edits. Ask for the source project to edit maps.

Important for your Legends work: the live Legends button reads GitHub. It cannot see edits sitting only in your local Porymap folder. If you have those edits, import a pack made from that folder.

Quick help is always nearby: tap **? Guide** in the header (including on narrow screens), or press **F1**. The quick guide opens over your work. Closing it restores the same workspace. Event, export and review dialogs also offer a small guide link. The full field guide opens separately.

Install or open

The TileForge home page offers **Open Web App** and **Download APK**. In a browser, open the web app. On Android, download the APK, open it through your file manager and follow Android's installer prompts. If Android requests it, allow that particular browser/file manager to install this app. Future updates should install over the same app rather than requiring you to clear its data.

The web app and installed APK have separate draft storage. Moving between them requires an exported editable project. For a first try, Help contains an optional **Offline practice island**. Your real source files are only changed after exported edits are integrated into the game project.

Make a source pack on a computer

Download the TileForge source and keep its tools folder available. With Python 3 installed, run:

```
python3 "/path/to/tileforge-studio/tools/pack_project.py" "/path/to/legends" "legends-source.zip"
```

Replace both example paths with your folders. Transfer `legends-source.zip` to the tablet, then open it in TileForge. The pack excludes ROMs and saves, includes map/event/tileset sources, and preserves source paths. A full source ZIP is also accepted; use the smaller pack if the full archive is too large. The file picker limit is 350 MB; imported source contents have a separate 220 MB limit.

2. Load, orient yourself and make a tiny first edit

1. Open TileForge and stay on **1 · Load** until you choose a project or a draft.
2. Choose **Emerald: Legends** for the public project. A new import starts in **Littleroot** when that layout exists. A saved project/draft keeps its previously selected map.
3. Choose **Map studio**. On a wide screen, maps appear in the left explorer. On a compact screen, use the searchable map dropdown above the canvas.
4. Tap **Fit** to frame the map. Use **+ / -** to zoom. Select **Move** to drag the canvas with one finger without painting. Fit resumes automatic fitting after a rotation or resize.
5. Choose **Tiles** in the palette. In **Collision & display**, leave **Paint collision** and **Paint elevation** off for an artwork-only edit.
6. Choose **Pick**, then tap an existing ground cell. The tile becomes your brush, and the status readout shows X/Y, tile, collision and elevation. Coordinates start at zero at the map's top-left corner.
7. Choose **Paint**, tap one nearby cell, then tap **Undo** to restore it. This confirms the basic editing loop before you make a larger change.
8. Tap **Save draft** and wait for **Draft saved**. Continue with the task below that matches your edit.

On narrow phones, the palette is behind **Tiles** in the toolbar. Selecting a brush closes the drawer and returns to the map. On compact displays, panels open over the canvas; **Done** tucks them away. The app adapts to available CSS viewport space, including system scaling and split-screen; device marketing resolution is not the same thing as that usable space.

3. Find the right tile and paint the map

A **metatile** is the 16 × 16 unit you place on a map. The thumbnail is enlarged for your fingers; this does not enlarge the actual game art. A **tileset** supplies many metatiles. A map uses one primary set and one secondary set.

Palette control	What it does
P / S, source legend and category subheaders	Identify the actual primary or secondary tileset supplying the tile.
Tileset source dropdown	Show both sets, only the primary, or only the secondary.
Tile category dropdown	Browse a type such as Cliffs, Water, Signs or Furniture & interiors. Search inside the dropdown. Empty categories are hidden.
Search tiles	Search names, categories, behaviors, tileset names, location usage or IDs. Try `water`, `Petalburg` or `#01B`. Filters combine.
Pinned	Show brushes you starred. Select a brush and tap the star beside its source to pin/unpin it.
Used	Show metatiles present on the active map. This counts map cells, not the border or other maps.
Recent	Show the last 16 selected brush identities. This is a filter, not a chronological history panel.
Clear filters	Restore the full primary/secondary palette if a tile seems to have disappeared.
Selected tile → Tile group → Group selected tile	Override a tile's category. Choose **Automatic group** to remove your override.

The ID on a thumbnail is its combined map ID, written in hexadecimal. Selected tile details also show its local ID within its own source. Matching numbers in different tilesets do not guarantee matching artwork.

Automatic groups use descriptive source labels, then behaviors, plus a limited artwork review for the exact inspected Littleroot pair. Uncertain tiles stay in **Other**. Your custom grouping has priority. Grouping changes palette organization; it does not change collision, elevation, interactions or game files. Pins and groups travel in editable project ZIPs and drafts, rather than the changed-source ZIP.

Paint, fill and sample

Select a metatile, choose **Paint**, then tap or drag across the map. One completed stroke becomes one Undo entry. **Fill** replaces a connected region matching the selected field: in art mode this is the same metatile ID, even where movement values differ. Art painting/fill preserves existing collision and elevation. **Pick** samples a cell, including its movement readout; select Paint again to place the brush.

A larger object such as a house is made of several metatiles. Place the matching pieces individually. Multi-tile stamping, smart paths and automatic edge joining are suggested future features, not current controls.

Use tiles from another location

Each location opens its own saved primary and secondary tilesets. Changing maps clears palette search, category/source and quick filters and resets palette scrolling. The location picker shows the active map. This does not revert an edited tileset assignment: saved source references remain authoritative. You can still search other locations, switch either set, scroll the entire active pair, or apply any category/source filter.

1. Save a draft or editable project before experimenting.
2. Open **Map tilesets**.
3. Search **Use pair from location** to choose an existing location's compatible primary/secondary pair. Alternatively, search **Primary tileset** or **Secondary tileset** to replace just one half.
4. Review the whole map and the border. Existing cell IDs remain the same, so changing a set can change existing artwork throughout the layout. Collision/elevation stay intact.
5. If cells turn magenta, choose available replacement tiles or Undo the swap. Changed-source export is blocked if a changed tileset pair still leaves missing tiles. An editable project can still preserve the unfinished work.

This changes the active layout's references; it does not copy one isolated metatile or add a third tileset. Other source maps sharing this layout also share its visual map data. Location details show where a shared set is used, rather than claiming it belongs to a single town.

4. View and edit collision and elevation

Collision controls passability. **Elevation** controls which level connects to another. A picture of water, a bridge or a cliff is not enough by itself to set the correct movement.

1. Open **Collision & display**, enable **Movement overlay**, and adjust **Opacity** so the art remains visible. Viewing the overlay alone does not edit anything.
2. Choose **Pick** and tap a working nearby cell to inspect its collision/elevation. For routine edits, copy the values used by the same feature nearby.
3. Choose **Movement** in the palette for 32 common presets. The left column is walkable; the right column is blocked. The displayed character is elevation in hexadecimal.
4. Select a preset, switch to **Paint**, then paint the relevant cells. The preset enables both movement fields; artwork is retained.
5. For a change to only one field, use **Paint collision** / **Paint elevation** in Collision & display. Enable only what you intend to change; set collision 0-3 and elevation 0-15.
6. Choose **Tiles** when finished, and check the two paint toggles before returning to artwork edits.

Common elevation	Typical purpose
0, walkable	Transition between different levels, commonly on stairs.
0, blocked	Common level-independent obstacle setting.
1	Surf-related level. Copy an existing working shore/water arrangement.
2-E	Ordinary distinct elevation levels. Many outdoor paths use 3, but inspect the actual map.
F	Multi-level bridge behavior; preserves the player's approach level.

These describe the conventional Gen III movement model; the game's engine/scripts and metatile behaviors still determine the result. Do not flatten a cliff or bridge to one value just because it looks connected. Keep raw collision values 2/3 when they already have a project-specific use. A blocked overlay is red, and walkable labels are white. Movement fill follows only the enabled movement fields and can cross different art tiles.

When you export, request a game test that walks the changed route from both directions and checks stairs, surf edges and bridges. An overlay is an editing aid, not a gameplay simulation.

5. Change the map border or tileset pixels

Repeating map border

Open **Map border** above the palette, choose a metatile, then tap the small border-pattern cell you want to replace. Enable **Show around map** to preview the repeated pattern. Undo restores border painting. Source exports include the layout's exact declared border file. The common Emerald pattern is 2 × 2, but declared supported dimensions are retained.

This edits the repeating border pattern. It does not draw connected neighboring maps or resize the map/border. A shared border file affects every layout referencing that file.

Pixel studio: change the artwork itself

1. Save an editable project before a shared graphics change.
2. Open **Pixel studio**. Choose the primary/secondary **Sheet** and the appropriate display **Palette**. This palette selector changes the preview, not palette colors in the game.
3. Zoom in, choose a color index from 0 to 15, then paint a pixel. Pick samples a pixel's existing index. Color 0 is transparent in upper metatile layers.
4. Return to Map studio to inspect the change. Open another layout using the same graphics to check how the shared edit looks there.
5. Undo if necessary. The exported change is the indexed PNG at its original source path; the palette indices and indexed bit depth are retained.

Pixel studio edits an existing graphics sheet, not metatile composition. It does not import new tiles, rearrange metatile layers or edit palette colors. The preview uses static daytime palettes; seasonal transformations, animations, lighting overlays and NPC movement run in the game. The narrowly reviewed artwork categories only apply while their exact inspected graphics/metatile pair still matches; changed artwork can fall back to source-based groups or Other. You can tag it yourself.

6. Add or edit NPCs, signs, triggers and warps

1. Open **NPCs & events**. Confirm **Event map** is the intended source map. Several maps can share a layout, while their events remain separate.
2. To edit an existing event, tap its marker or choose it in **Select event**. Markers are **N** (NPC), **S** (sign), **T** (trigger) and **W** (warp); these are source markers rather than animated character sprites.
3. To add an event, tap **+ NPC**, **+ Sign**, **+ Trigger** or **+ Warp**, then tap the desired map cell. On compact screens the controls tuck away so you can place it.
4. Set X/Y and elevation. For an NPC, choose appearance and movement; set movement ranges only if needed. A visibility flag of 0 means no hide flag. Existing story flags should stay unchanged unless you intentionally want different story behavior.
5. For NPCs/signs/triggers, keep **Existing script** when preserving the original interaction. For a simple new conversation, choose **Dialogue text**, enter standard ASCII text and press **Save event**. Existing script blocks remain intact; TileForge creates its own separate block. Changing an existing NPC's interaction to dialogue can replace its story/trainer interaction even though its other metadata is preserved.
6. Tap the saved marker to reopen it and verify the fields. Undo rolls back map metadata and generated dialogue together.

A sign also needs a suitable sign metatile on the map; placing an S marker does not paint the sign artwork. A trigger additionally needs its required variable and value. A warp needs the correct destination map and destination warp **index** (the source list is zero-based). Leave unfamiliar constants alone and describe your intended behavior when sharing edits.

Dialogue is wrapped for the game's text box, with a 1,500-character limit. Use standard letters/punctuation; $, backslash, braces and non-ASCII characters are reserved. This is not a general script editor. New events append so existing indices stay in place. Specialized clone NPCs and hidden-item/weather events are displayed but cannot be edited; event deletion is not included.

Return to tile painting: close event mode. On compact screens, **Done** keeps event mode active while hiding its controls; tap the highlighted NPCs & events header to reopen the controls, then close the panel. Use Move at any time to pan.

Review events before sharing

Open **NPCs & events → Review events**, or choose **Review events before sharing** in Export. It checks the active source map and changed loaded maps. For live Legends, it also tries to load their local scripts and named warp destinations. Source packs are checked using their available files.

The review covers numeric event bounds, duplicate NPC local IDs, changed overlapping placements, changed NPC collision/elevation and movement ranges, loaded script labels, incomplete TileForge dialogue blocks, and warp indices in loaded destinations. A finding can describe an intentional story setup. It never blocks export or alters files merely by running.

- **Open event** takes you to the normal event editor.
- **Preview correction** shows the exact field and before/after values when a bounded correction is available. **Apply this correction** changes only that event field and adds an Undo entry. Review again after other edits; stale corrections are refused.
- Overlaps, script identities, story flags and warp destinations require your judgment. The app will not automatically renumber NPCs, pick a different destination or rewrite arbitrary story scripts.
- Expand **Files and references not verified** for missing or external dependencies. “No issues found” only describes these available checks. It is not proof the game is bug-free.

Optional local AI: expand **Local AI assistant** and tap **Download / load local AI**. The first load downloads approximately 390 MB of pinned SmolLM2 360M and Transformers.js/ONNX runtime assets from Hugging Face and jsDelivr. Use Wi-Fi and allow several minutes and enough device memory. The model runs on your device in a worker, using CPU/WebAssembly; no API key, AI server or project upload is used. Cached downloads can work offline while browser storage retains them. **Stop / unload** cancels loading/generation and frees the worker; **Remove AI downloads** clears its downloaded cache.

After loading, tap **Ask local AI** on a finding for an explanation and suggested manual test. This small general model is experimental; it is not trained on your complete game and can be wrong. Deterministic source checks supply the evidence and offered field corrections. Model text cannot execute code or apply edits. Neither path simulates arbitrary scripts or story progression. Build and playtest NPC interaction, flags, warps and return trips before keeping the result.

7. Save drafts, undo mistakes and make backups

Save option	Use it for	What to remember
Save draft / automatic draft	Pausing and resuming on the same browser/app	Keeps original files and undo/redo history. Wait for Draft saved. Drafts are local to this app/origin, not cloud sync.
Export → Save editable project	Portable source backup or moving between web and APK	Keeps loaded source files, original baselines, chosen map, pins and category tags. Reopening starts a new undo history. Live-source exports contain only maps/assets already loaded; more can load online from the pinned revision.
Export → Share edits ZIP	Sending source changes for review/integration	Contains changed files and a hash manifest. It is not a complete workspace or a ROM.
Export edited GBA in ROM mode	Keeping your manual ROM changes	Creates a new ROM copy. Source projects cannot use this route to compile a ROM.

Automatic drafts run after changes. Use the explicit button and inspect its success label before closing the app. Clearing browser storage, uninstalling the app or changing origins can remove access to drafts. Export a portable project at meaningful milestones.

The labelled **Undo** button shows the number of available edits; **Redo** restores an undone edit. Undo spans map, collision, border, tileset pixel and event changes. Pins and category tags are editor preferences and do not add game-file Undo entries.

Keyboard shortcuts, when focus is outside an input or dialog: **B** Paint, **F** Fill, **I** Pick, **H** Move, **Ctrl/Cmd+Z** Undo, **Ctrl/Cmd+Shift+Z** Redo, **Ctrl/Cmd+S** export an editable source project (or a ROM copy in ROM mode). Ctrl+S is a portable export, not Save draft.

After a set of edits has been integrated, open a fresh source pack or a fresh live session based on that integrated revision before starting the next batch. An old draft intentionally keeps its old baseline; re-exporting cumulative changes from it can conflict with files already integrated.

Delete old drafts

Choose **Load**, find the saved draft and tap **Delete**. The confirmation identifies the project and timestamp. Deleted drafts stay on this device and can be restored; deleting is not a portable backup or a way to clear their storage. Export a portable project first if you need a backup. Confirming moves that device copy to **Deleted drafts**, where **Restore** recovers it with its original baseline and undo history; it does not delete exported files or the open workspace. If the deleted draft was open, pending autosave is cancelled and the app does not immediately recreate it. Your next edit or explicit **Save draft** creates a new draft. Other drafts keep their own identity and baselines.

8. Export and send the required files back to chat

1. Press **3 · Export**.
2. Open **Review changed files**. For a map edit you normally see its `map.bin`; a border edit adds `border.bin`; a tileset swap changes `data/layouts/layouts.json`; pixels change a `.png`; events change the map's `map.json`, and new dialogue also changes `scripts.inc`. Actual paths follow your source project.
3. If an unrelated file appears, go back and inspect/Undo the accidental change.
4. Choose **Share edits ZIP**. In the browser, the next dialog is **Save to your device → Save file**. In the APK, use Android's Save As picker. Finish that save action so the ZIP actually exists on the device.
5. Also save an **editable project** for your own backup.
6. Attach the edits ZIP to chat. Include the note below. Attach the editable project too if your custom categories/pins or full working context matter.

Suggested handoff note:

```
Project: Emerald: Legends
Started from: live source / my local source pack
Map/layout: LittlerootTown_Layout
Changed area: X __ to __, Y __ to __
Intent: describe the visual or movement change
Also changed: collision / border / pixels / events / none
Local Porymap edits to preserve: describe them
Please compare the manifest baselines, show the diff, integrate
only matching files, then build and test the changed area.
```

A screenshot can help explain intent, but the ZIP supplies the exact files and original hashes. You do not need to upload a ROM for source integration. The app does not push to GitHub, update the game repository or publish a game release when you press Export.

9. Integrate, build and check the result

If you hand the ZIP back to chat, the integration work can be done there. If doing it yourself, keep a backup/branch of your current game checkout and use the helper from the **TileForge** repository:

```
python3 "/path/to/tileforge-studio/tools/apply_changes.py" "TileForge-edits.zip" "/path/to/legends"
```

This is a dry run. It compares every original file hash and checks the changed bytes before writing anything. If all files match, apply:

```
python3 "/path/to/tileforge-studio/tools/apply_changes.py" "TileForge-edits.zip" "/path/to/legends" \
  --apply
```

Original files are backed up under `.tileforge-backups/<timestamp>/` in that game checkout. Inspect the source diff, including paired map/script edits. If you see **CONFLICT: original file changed**, stop and reconcile the current files against the exported baseline; do not overwrite the current file or remove the manifest's check. This is how your other Porymap work is protected. The preflight catches source mismatches; the helper is not a version-control merge tool.

With the Legends build environment already installed according to its INSTALL.md, run the appropriate build in the **Legends game folder**:

```
make release
```

The Legends README documents `make release` for the normal player build and `make` for the Debug/developer build. Use its current build pipeline and output paths; TileForge itself does not install the compiler or build the ROM. Source editing does not require the TileForge APK to become an emulator.

Test the built result: inspect the whole edited area, walk from each approach, try changed doors/warps, read signs, talk to NPCs and verify shared graphics in another affected map. For bridges/stairs/water, test the relevant elevations and shore crossings. Keep your original game/save backup and test a copy. A successful compilation alone does not prove the movement or event logic is correct.

10. Troubleshooting and next improvements

What you see	What to do
A tile seems missing from the palette	Press Clear filters; check the active P/S source and category. It may be in Other.
A category looks wrong	Select the tile, open Selected tile and set Group selected tile. Save a draft/project. Report the source name and #ID for a future automatic fix.
A category vanished after editing pixels	Reviewed artwork annotations require exact inspected bytes. Your source labels/behaviors and manual tags still work.
Magenta map or border cells	The active pair lacks those metatile IDs. Replace them or Undo the tileset swap before sharing.
Touching the map will not paint	Choose Paint, turn off event mode, and confirm you are in Map studio with Tiles selected.
Artwork stays unchanged while painting	Check Paint collision / Paint elevation. Movement tools intentionally retain artwork.
The map looks wrong in Pixel studio	Try the palette actually used by the metatile. A single sheet can be referenced with several palettes.
An imported pack cannot open a map	It may lack that map or custom assets. Make a complete source pack from the actual working checkout.
Live Legends cannot load	Check connectivity or open a previously exported project/source pack. Live assets need network access.
A draft does not appear on another device	Drafts are local. Export/import an editable project to transfer work.
The game ROM opens without maps	Open source files for map editing. Automatic ROM map detection and decompilation are not included.
Export says no changed files	Category tags/pins alone are not game edits, or all game edits were undone. Save an editable project to keep preferences.

Ideas worth prioritizing

These are proposals, not features included in 0.4.2:

1. **Before/after export preview and affected-map list.** Show changed cells and which maps share an edited graphics/border/layout file before integration. This would make shared asset edits easier to review.
2. **Multi-tile stamps and smart edges.** Copy a tree, house or cliff segment as one brush, then add optional path/shore edge joining. Porymap's prefabs and smart paths are useful references.
3. **Batch category review and portable category packs.** Select several unknown tiles, assign a group once, and share a small versioned metadata file without sending the whole project. Keep provenance and exact asset checks.
4. **Draft names and checkpoints.** Let you name milestones, compare them and return to a checkpoint, while keeping the current Undo behavior.
5. **A focused gameplay check after integration.** Complement Review events with a local test build that checks movement, flags and return warps in the actual game. This needs game/build tooling beyond the source checker.

The highest-value next feature is the before/after export preview: it helps you verify exactly what you will send and catches unintended shared changes early.

Sources and scope

This guide follows the TileForge 0.5.0 source and its actual UI labels. Category audit: CATEGORY-AUDIT.md. The editor is an independent alpha inspired by Porymap and Hex Maniac Advance.

Reference reading: Porymap's map tiles and borders, collision model, and tileset label/behavior documentation. Build commands: Legends README. Porymap's documented capabilities are references, not claims that TileForge implements all of them.

Browser viewport checks and automated source/binary tests support this release. Physical Android installation, file-picker/export behavior, Fold/Legion Go hardware and in-game behavior need testing on those devices/builds. Source previews are static daytime views. No map resizing, metatile assembly editor, animation playback, arbitrary script editor, compressed ROM graphics reinsertion, general ROM map detection or automated decompilation is included.

Asset and local AI sources

The Fuecoco app sprite is the front sprite from the pinned Emerald: Legends source, based on RHH/pokeemerald-expansion; see its contributors and sprite credits. Pokémon/Fuecoco belong to their respective rights holders. The local assistant uses HuggingFaceTB's SmolLM2, the ONNX Community conversion and Transformers.js. These assets do not change ownership of game artwork.